

AN OVERVIEW OF PATTERN-DIRECTED INFERENCE SYSTEMS

D. A. Waterman and Frederick Hayes-Roth

August 1978

P-6193

The Rand Paper Series

Papers are issued by The Rand Corporation as a service to its professional staff. Their purpose is to facilitate the exchange of ideas among those who share the author's research interests; Papers are not reports prepared in fulfillment of Rand's contracts or grants. Views expressed in a Paper are the author's own, and are not necessarily shared by Rand or its research sponsors.

**The Rand Corporation
Santa Monica, California 90406**

ABSTRACT

Pattern-directed inference systems are programs that look for interesting or important situations occurring as patterns in the input or memory data to direct their activity. These patterns select pieces of program code to be executed. In this paper a brief overview of pattern-directed inference systems is presented, including an historical perspective, a review of basic concepts, and a survey of current work in this area. Examples of the best known type of pattern-directed inference system, the production system, are presented and discussed in detail.

BASIC CONCEPTS

During the past few years researchers have become interested in topics at the intersection of artificial intelligence, cognitive psychology, and pattern recognition. One focus for this interest has been a very simple but powerful idea: a program organization based on data or event-driven operations. The program responds directly to a wide range of data or events (possibly unanticipated) rather than operating on expected data in known formats using a prespecified and inflexible control structure as found in conventional programs. Such an organization can be characterized as pattern-directed: Patterns occurring in the data select pieces of code in the system to be activated. Systems organized this way have been used primarily for deductive inference and, to a lesser extent, for inductive inference or learning. We shall call a system that exploits patterns in the input data to select and trigger activities a pattern-directed inference system (PDIS). A PDIS has three basic components: (1) a collection of substructures called pattern-directed modules (PDMs) which can be activated or "fired" by patterns in the data, (2) one or more data structures that may be examined and modified by the PDMs, and (3) an executive (also called an interpreter) that controls the selection and activation of the PDMs (Hayes-Roth, Waterman, & Lenat, 1978). Examples of PDISs are production systems, grammatical inference systems, and any deductive or inductive inference system that uses patterns or rules to guide the

decisionmaking process.

An important type of PDIS is the rule-based system (RBS) in which the PDMs consist of antecedent-consequent pairs called rules. The distinguishing characteristic of a rule-based system is the separation of data examination from data modification. The examination of data generally takes place in the antecedent or left-hand side of a rule, while data modification is normally handled by the consequent or right-hand side. No further assumptions are made concerning the representation of either antecedents or consequents, the structure of the data they manipulate, or the type of control structure determining how rules make contact with data.

Data examination consists of comparing the patterns associated with the antecedents to the elements in the data structures. The patterns may be defined in many ways: as simple strings, complex graphs or semantic net structures, or even arbitrary segments of code capable of inspecting data elements. Data modification can be as simple as asserting that a certain proposition is true or as complex as firing arbitrary actions to modify data, rules, or the environment (via output operations). Information in the data structures can be in the form of lists, trees, nets, rules or any other useful representation. Even the control structure can range from a simple implicit one, as used in many production systems, to a complex explicit one based on special heuristics or metarules.

Production systems are a well-known type of rule-based

system in which the control structure can be mapped into a relatively simple recognize-act paradigm. A production system typically consists of a data base, a set of production rules of the form antecedent -> consequent, and an interpreter that embodies a mechanism to examine the data and determine which rules to fire.

Production systems most often have antecedents comprising conditions that characterize the data base and consequents comprising actions that manipulate the data base. The execution of this type of production system can be defined as a series of recognize-act cycles as diagrammed in Fig. 1. Each cycle involves deciding which rule to fire and then executing the actions associated with that rule. Since the effect of executing the actions may change the data base, different rules may fire on successive cycles.

The recognize portion of the cycle consists of comparing rule conditions to the data base to search for matches. The conflict set comprises all rules that are satisfied by the current data. In Fig. 1, a rule matches the data base if all the Cs in its antecedent are found in the data base. Then conflict resolution, the process of deciding which satisfied rule to fire, is applied to the conflict set. Once a rule is selected the cycle concludes with the act step during which the actions in the consequent of the rule are executed.

R E C O G N I Z E

DATA BASE: C5 C1 C3

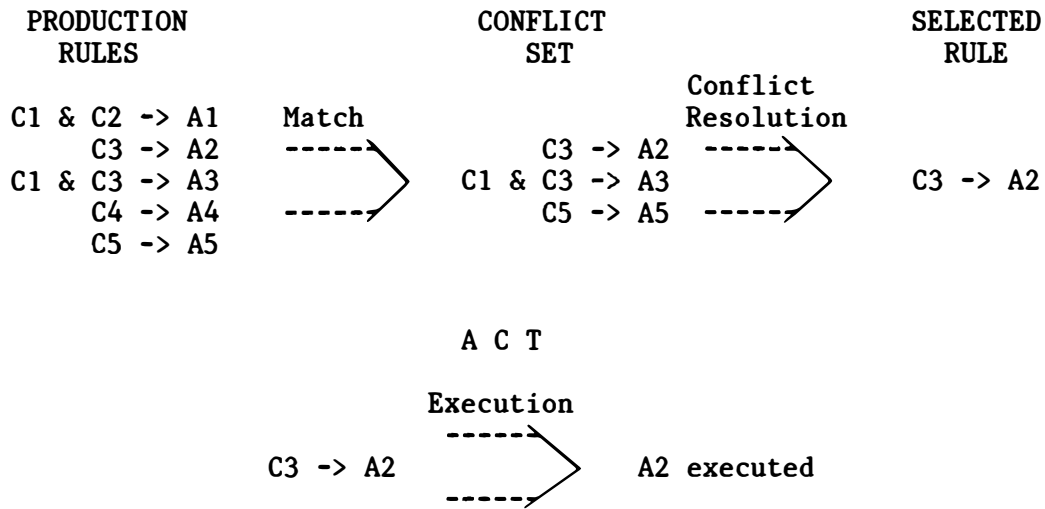


Figure 1. Production system recognize-act cycle.

The history of the production system is summarized succinctly in Newell and Simon (1972):

The production system was one of those happy events, though in minor key, that historians of science often talk about: a rather well-prepared formalism, sitting in wait for a scientific mission. Production systems have a long and diverse history. Their use in symbolic logic starts with Post (see Post, 1943, from whom the name is taken). They also show up as Markov algorithms (Markov, 1954). Their use in linguistics, where they are also called rewrite rules, dates from Chomsky (1957). As with so many other notions in computer science, they really entered into wide currency when they became operationalized in programming languages, first in string manipulation systems (COMIT, see Yngve 1958; and SNOBOL (Farber, 1964)) and in compiler translation languages (Floyd, 1961).

One of the first uses of production systems in psychological modeling was the work of Newell and Simon

(1965) in chess analysis and in the analysis of cryptarithmic problems (Newell, 1967, 1968). In both cases the intent was to use production systems to model human problem solving in specific task areas. The basis for the research was a method of protocol analysis (see Waterman & Newell, 1971; Newell, 1967) in which subjects were instructed to "think aloud" as they solved a problem. The resulting protocol or verbalization of their problem-solving effort was used to infer productions that could account for the behavior noted in the protocol. Newell and Simon (1972) consider the production system to be a good candidate for modeling human cognitive processes for a number of reasons, the most important being:

- * They have the computational generality of universal Turing machines, providing the models with a wide range of potential information processing capabilities.
- * Since production rules tend to represent independent components of behavior, the creation and addition of rules can be incremental; i.e., new rules can be added one at a time.
- * Under certain assumptions, the production system data base models the functional characteristics of human short-term memory, and the production rules provide a possible model of human long-term memory.

Production systems usually represent antecedents as logical combinations of predicates whose truth values can be determined by examining the data. Consequents are usually represented either as procedures to alter the data, rules, or environment, or as assertions about the truth value

of statements. The production system illustrated in Fig. 1 is antecedent-driven, i.e., during the matching process it makes initial contact with the data through the conditions in its antecedents. Rule-based systems that make initial contact with the data via the consequents have also been termed "production systems." These consequent-driven RBSs have rules analogous to logical implication statements. Thus, $A \& B \& C \rightarrow D$ in such a system would be interpreted as "if A, B, and C are true then D is true." Consequent-driven behavior involves searching for preconditions to justify or confirm the consequents.

STRUCTURE OF PATTERN-DIRECTED INFERENCE SYSTEMS

Contemporary PDISs have evolved from a humble beginning, the Post production systems (Post 1943; Minsky, 1967). Post's symbol manipulation systems were composed of grammarlike rules called "productions" used for specifying string replacement operations. A rule in a Post production system would typically have the form $AxB \rightarrow AyB$, stating that an occurrence of the string x in the context of A and B would be replaced by the string y. Imposing an implicit control structure on a collection of ordered string replacement rules leads to the concept of the Markov normal algorithm (Markov, 1954; Galler & Perlis, 1970). In a Markov algorithm the rules are tested against a data base that is a simple string of symbols, and the first rule found to be applicable is applied, resulting in an alteration of the string. After that

rule is applied, testing resumes, beginning with the highest priority rule. This test-apply cycle is repeated until no rules are applicable. Figure 2 illustrates a Markov algorithm that transforms any string composed of a's and b's into the string AB (Waterman, 1976b).

Alphabet: a, b, A, B
Variables: x
Productions: (1) aa -> a
 (2) bb -> b
 (3) xa -> ax
 (4) a -> A
 (5) b -> B

Figure 2. A simple Markov normal algorithm for mapping strings of a's and b's into the string AB.

The productions are arranged in order with highest priority first. After any production is applied, testing begins again with rule 1. If the data base is the string abaaba, rules 1, 3, 1, 2, 3, 1, 4, and 5 are applied in that order to produce the string AB. Note that the highest priority productions must be applied first to insure that the algorithm will terminate and give the correct result.

Production systems can be thought of as a generalization of Markov normal algorithms. The early production systems for modeling human cognition (Newell, 1972; Newell & Simon, 1972; Newell, 1973; Newell & McDermott, 1975; Hunt & Poltrock, 1974; Klahr, 1973) have a simple form: A small data base or working memory contains a collection of active memory elements

(symbols or lists), and a long-term memory contains condition-action rules selected for execution by comparing rule antecedent with the elements in memory. The rule consequents are actions that add, delete, or modify memory elements. Figure 3 illustrates a production system of this type that performs subtraction ($P - Q$) by iterated counting operations.

Data Base: (START) (P 5) (Q 3)

Variables: x1 x2

Rules:

1. (START) $\rightarrow$ delete((START))
deposit((COUNT 0))
2. (COUNT x1) & (P x2) & (Q x2) $\rightarrow$ say(x1)
stop()
3. (COUNT x1) & (Q x2) $\rightarrow$ replace(x1,s(x1),(COUNT x1))
replace(x2,s(x2),(Q x2))

Actions:

- delete(b): removes b from the data base.
- deposit(b): puts b into the data base.
- replace(b,c,d): replace b with c in data element d.
- print(b): prints b at terminal.
- stop(): stops production system execution.
- s(b): returns the successor of b (i.e., b+1).

Figure 3. A production system for performing the integer subtraction, P minus Q (see Fig. 4).

The highest priority rule (topmost) matching the data base fires on each cycle. Thus rule 1 fires first, deleting the memory element (START) and adding the element (COUNT 0). Next, rule 3 fires repeatedly, incrementing the numbers associated with COUNT and Q until the numbers associated with P and Q are equal, causing rule 2 to fire. The x's are variables that are instantiated during the match process and evaluated during the execution of the actions. Note

that the two occurrences of the same variable (x2) in rule 2 mean that the condition is satisfied only when the numbers associated with P and Q are identical.

```
Data Base:  OBJECT start;   OBJECT p      OBJECT q
              value IS 5;   value IS 3;

Rules:  Rule 1:  IF:  THERE IS A start
              THEN: DELETE THE OBJECT start
                   & CREATE A count WHOSE value IS 0;

          Rule 2:  IF:  THERE IS A count
                   & THE value OF p IS THE value OF q
              THEN: SEND THE value OF count TO THE USER
                   & RETURN SUCCESS;

          Rule 3:  IF:  THERE IS A count
                   & THERE IS A q
              THEN: SET THE value OF count TO 1 + value OF count
                   & SET THE value OF q TO 1 + value OF q;
```

Figure 4. A RITA production system for performing the integer subtraction, P minus Q (see Fig. 3).

Of course, production systems are not restricted to the use of simple matching techniques on a list-structured data base. The matching process can evaluate arbitrary predicates on the data base. Similarly, the action components can specify a variety of executable procedures. To illustrate the contrast, the production system of Fig. 3 has been written in RITA, a high-level production system language (Anderson & Gillogly, 1976a, 1976b; Anderson, 1977; Anderson et al., 1977). The RITA equivalent of Fig. 3 is shown in Fig. 4. Note that the two systems are isomorphic and have their rules executed in exactly the same order. In RITA the data base is a collection of object-attribute-value triples, rather than an arbitrary list structure,

and reserved words (shown in capital letters) abound. Also, the LHSs explicitly state how they should be evaluated relative to the data base, while in the example of Fig. 3 this match or evaluation is implicit.

Figures 2, 3, and 4 illustrate the operation of antecedent-driven production systems where matching is based on the antecedent properties. In consequent-driven systems, however, matching is based on rule consequents. A consequent-driven system is usually given a premise to "prove" through deductive inference. The consequents of the rules are examined to find those which could confirm the given premise. From this set of rules one is chosen and examined to see if all the premises in its antecedent are true relative to the data base. If they are, the rule is fired and the given premise is proved. If they are not, the process continues recursively in an attempt to deduce the validity of each premise in the antecedent of the rule. This process of working backward through the rules from consequents to antecedents in search of a causal chain that will prove the given premise is called backward chaining or consequent reasoning.

An abstract example of this type of system is shown in Fig. 5. The letters represent data base elements and are considered true if in the data base. In this simple system, the action of firing a rule inserts the right-hand side letter into the data base.

DATA BASE: B C

RULES: (1) B & D & E $\rightarrow$ F
(2) D & G $\rightarrow$ A
(3) C & F $\rightarrow$ A
(4) C $\rightarrow$ D
(5) D $\rightarrow$ E
(6) A $\rightarrow$ H

Figure 5. A simple consequent-driven production system.

Assume the given premise is that "H is true." Since H is not in the data base it must be deduced by backward chaining through the rules. The system first chooses rule 6 and attempts to show that A is true, since A implies H. Unfortunately A is not in the data base and must itself be deduced. This can be attempted through either rule 2 or 3. Assume the system first tries rule 2. It must now show that both D and G are true. D is easy; it can be inferred from rule 4. G is another matter; nothing can be inferred about G since no rules contain G on the right-hand side. The system now backs up and tries to infer A through rule 3. This time it succeeds since C is true and F can be inferred by showing that B, D, and E are true. A causal chain has now been produced that proves the given premise "H is true," as shown in Fig. 6.

Pattern-directed inference systems, in general, are characterized by a separation of data, rules, and a control schema. The control schema can be characterized as having four basic parts: selection, matching, scheduling, and execution, as shown in Fig. 7.

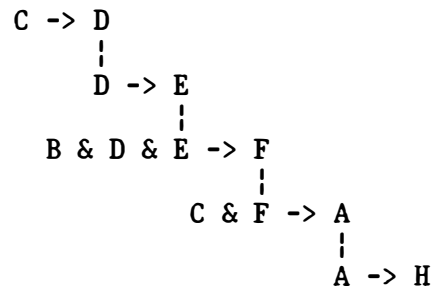


Figure 6. Causal chain for inferring H from B and C, using the data base and rules of Fig. 5.

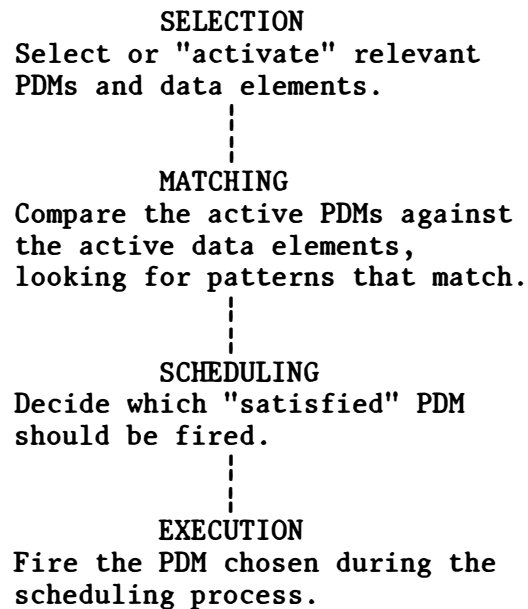


Figure 7. The control schema of pattern-directed inference systems.

Selection consists of deciding what subset of procedural task knowledge (PDMs) and declarative state knowledge (data or working memory elements) should be considered for use during the next cycle of operation of the system. In production systems selection may be trivial, e.g., on each cycle all rules and all

data elements can be considered, or may be quite complex, e.g., special filters (McDermott, Newell, & Moore, 1978) can be designed to eliminate from consideration many rules that could not possibly match the current data. In sophisticated rule-based systems selection may involve the use of hierarchies, state transitions, metarules, or complex control schemata like Petri nets to narrow the set of rules considered.

The process of selection can take many forms. It may involve tagging rules as new data elements are added and deleted, e.g., in Fig. 8 if C3 were added to the data base, rules 2 and 3 would be tagged and would remain tagged until C3 was deleted. Selection in this case involves choosing the set of tagged rules (McDermott, Newell, & Moore, 1978).

```
(1) C1 & C2 --> A1
(2) C2 & C3 --> A2
(3) C3 & C4 --> A3
(4) C4 & C5 --> A4
(5) C5 & C6 --> A5
(6) C6 & C7 --> A6
```

Figure 8. A single level production system

Alternatively, rules may be partitioned into sets or hierarchies, and selection may consist of applying metarules to choose one particular rule set. For example, suppose set A contains rules 1 and 2 from Fig. 8, set B contains rules 3 and 4, and set C contains rules 5 and 6. The metarules might state that if condition X exists set A should be selected, otherwise if Y exists B should be selected, otherwise set C should be

used. Selection is determined by evaluating these metarules. The same effect could be achieved by arranging the rules into a hierarchy as shown in Fig. 9. In this and subsequent figures, the minus symbol before a data element (e.g., -X) in a condition denotes that the data element must be absent from the data base for the condition to be satisfied.

```

P1:   (0)  -X &  Y  --> P2
      (1)  C1 & C2 --> A1
      (2)  C2 & C3 --> A2

P2:   (0)  -X & -Y  --> P3
      (1)  C3 & C4 --> A3
      (2)  C4 & C5 --> A4

P3:   (0)           X  --> P1
      (1)  C5 & C6 --> A5
      (2)  C6 & C7 --> A6

```

Figure 9. Hierarchical production systems.

Here the actions P1 and P2 mean that the named production systems are to be executed (selected), and each production system is assumed to be ordered with highest priority rules first. The rule sets could also be associated with nodes in a graph or net as shown in Fig. 10.

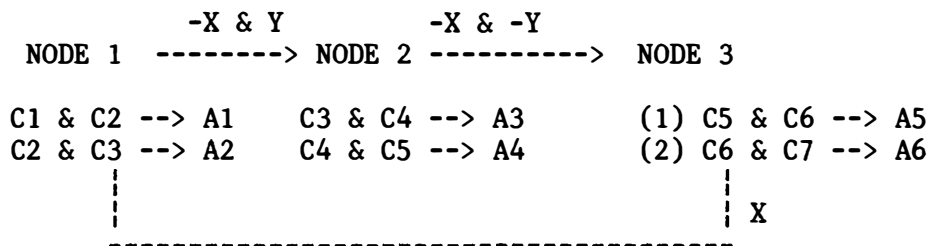


Figure 10. Transition network production system.

A selection mechanism similar to this but more complex, making use of Petri nets instead of finite state transition networks, is used by Zisman (1978).

Matching consists of searching the selected rules and comparing them against the selected data, looking for rules whose conditions are satisfied. The matching operation can be time-consuming, as it may involve instantiating many variables; this is one reason for using selection--it can significantly improve the overall efficiency of the system.

Once the satisfied rules are identified, the scheduling process (also called conflict resolution) decides which rule to fire. The scheduling mechanism may be implicit in the system or may be explicit as a set of metarules or procedures describing how to choose the rule to fire. Metarules permit the application of dynamic conflict resolution heuristics in a straightforward and understandable way.

Execution consists of firing the chosen rule, i.e., accessing and executing the procedures associated with the pattern elements that matched the current data. The result of execution is a modification of data elements or structure, or I/O operations.

EARLY WORK WITH PATTERN-DIRECTED INFERENCE SYSTEMS

Following the seminal work of Newell and Simon in the 1960s (see earlier references), production systems have attained

a prominent position in artificial intelligence and cognitive psychology research. One of the first uses of production systems in a purely artificial intelligence application was work on the representation and learning of heuristics in a game-playing environment (Waterman, 1968, 1970). Heuristics for generating favorable moves were represented as production rules and were learned over the course of playing a few games. This approach, which separates the problem of knowledge acquisition (theory formation) from the program that applies the theory, is reflected in the separation between Meta-DENDRAL (Buchanan, Feigenbaum, & Sridharan, 1972; Buchanan, 1976; Buchanan & Mitchell, 1978) a program designed to formulate rules of mass spectrometry, and Heuristic DENDRAL, a performance program that applies such rules to the analysis of molecular structure (Buchanan, Sutherland, & Feigenbaum, 1969; Smith, Masinter, & Sridharan, 1974).

In the early 1970s, the use of production systems as models of cognition was one focus of research at Carnegie-Mellon University. This work included models of small group interactions (Friend, 1973), cognitive development (Klahr, 1973; Klahr & Wallace, 1976; Young, 1973), human memory (Newell, 1972, 1973), symbolic imagery (Moran, 1974), and cognitive processes in writing computer programs (Brooks, 1975). More AI-oriented work included learning grammarlike productions for concept identification (Hedrick, 1974, 1976; Hayes-Roth, 1976a, 1976b), development of adaptive production systems (Waterman, 1975, 1976a), applications of production systems for speech

understanding (Hayes-Roth & Mostow, 1975, 1976; Hayes-Roth, Mostow, & Fox, 1977), and the use of production systems as AI programming languages (Rychener, 1976). At the same time, at Stanford University, Meta-DENDRAL was being pursued and MYCIN (Shortliffe et al., 1973, 1975; Shortliffe, 1976), a rule-based system for advising physicians regarding antimicrobial therapy, was developed.

More recently, the use of pattern-directed inference systems as a framework for knowledge organization in AI work has spread to new areas, such as program synthesis and natural language understanding. In the next section recent work in new areas will be reviewed.

CURRENT USE OF PATTERN-DIRECTED INFERENCE SYSTEMS

Much research is being undertaken currently within the PDIS framework. In fact, there has been a veritable explosion of interest in this formalism during the past few years as evidenced by the large number of recent papers generated on this subject. This work can be divided into six main areas: program synthesis, system architecture, knowledge acquisition, knowledge engineering, natural language understanding, and cognitive modeling. Each of these is discussed in turn.

Program Synthesis

In the area of program synthesis the problem of creating programs from examples of their traces is being investigated in

the domain of man-machine interface software (Waterman, 1978). Here the user creates "agents" (programmed procedures) to perform useful tasks by supplying an exemplary programming (EP) system with examples of the desired behavior. The EP system "looks over the shoulder" of the user and transforms the sample interaction between the user and the computer into a general procedure capable of performing that class of tasks in the future. The agents being created are represented as sets of rules to facilitate their incremental growth.

Program synthesis systems are being developed that produce conventional rather than rule-based programs, but organize their knowledge about how to construct programs as sets of rules. One example is PMB (program model builder), a module of the PSI program synthesis system designed to create programs from natural language descriptions and traces (Barstow & Kant, 1976; Green, 1976). PMB's knowledge about synthesizing programs is represented as a set of rules whose antecedents test the state of the current program fragment being generated and whose consequents modify that fragment or create new subgoals (McCune, 1977). Another example is a production rule mechanism for generating LISP code from examples of input-output pairs (Biermann & Smith, 1977). The information for coding recursive loops and branches in LISP is represented in production rule schemata. These general formats can be instantiated by user-provided information to yield a set of production rules pertinent to the problem at hand. When these rules are executed they generate the desired LISP code. PECOS, a

knowledge-based system for automatic program synthesis (Barstow, 1977a, 1977b), also represents its programming expertise via condition-action rules about programming. The target program is initially specified as an abstract algorithm, and through successive application of programming rules the specification is gradually refined into the specific code. In a program synthesis system called SYNSYS (Manna & Waldinger, 1977) high-level specifications are accepted in mathematical notation, and transformation rules are used to map this description into the desired primitive program. Production rules also have been used to drive a program transformation system where the goal is to transform high-level, inefficient program descriptions into efficient but lower-level code (Kibler, Neighbors, & Standish, 1977).

System Architecture

In the area of system architecture, a variety of languages for knowledge representation have been implemented that contain special implicit pattern-matching capabilities. These include PLANNER (Hewitt, 1971), QA4 (Rulifson, Derksen, & Waldinger, 1972), KRL (Bobrow & Winograd, 1977, Barstow et al., 1977), OWL (Szolovits, Hawkinson, & Martin, 1977), and TELOS (Travis et al., 1977). AMORD (deKleer et al., 1977) is a language for expressing pattern-evoked procedures that monitor a pattern-indexed data base. Rules are used to perform antecedent reasoning on assertions in the data base. Several production system languages have also been developed, including

PSNLST, a general programming language for AI applications (Rychener, 1976); RITA, an Englishlike rule-based language for building small, intelligent programs called "user agents" that couple the user to the computer and perform routine interactions for him (Anderson & Gillogly, 1976a; Anderson & Kline, 1977; Waterman, 1977b); OPS, a production system language for building large, instructable systems (Forgy & McDermott, 1977; Rychener & Newell, 1978); EPS (Brooks, 1977), an experimental production system language similar to PSG; and APRIL, a production system language for modeling problem solving in a CAI environment (Smith & Sleeman, 1977). Aspects of production system efficiency have also been investigated (Hayes-Roth & Mostow, 1975; McDermott, 1977a; McDermott, Newell, & Moore, 1978; Forgy, 1977; Zucker, 1978; Rosenschein, 1978; Rhyne, 1977).

In some cases, new system architectures have been designed to fit particular tasks. For example, AM, a rule-based system for the discovery of new mathematical concepts (Lenat, 1976, 1977a, 1977b; Lenat & McDermott, 1977; Lenat & Harris, 1978) uses over 200 heuristic rules that communicate via an agenda mechanism to discover set-theoretic concepts such as singleton sets and natural numbers. A system for modeling language comprehension has been developed that uses rules in the form of "trigger" patterns and associated procedures (Rieger, 1976; Rieger & Grinberg, 1977; Rieger & London, 1977). The trigger patterns are located in associative "trigger trees" that monitor the flow of information through a data channel and can "fire" to insert or modify data in the channel. A

system for modeling office processes has been designed that uses a Petri net to control the selection of production rules (Zisman, 1976, 1978). This permits modeling concurrent, asynchronous processes in a fairly efficient manner. All three of these systems are based on interesting architectures that are potentially extensible to a range of tasks beyond the original goals of the designers.

There are many examples of pattern-directed inference system architectures making clever use of deductive and inductive inferencing techniques. For example, Klahr (1978) uses a planning process to select promising rules for deductively inferring desired conclusions. Planning involves finding chains of rules that connect the assumptions with the goals. This is accomplished by searching forward from the assumptions and backward from the goals simultaneously, looking for rules that link forward and backward chains (see also Kellog, Klahr, & Travis, 1977). NASL (McDermott, 1977a; McDermott & Forgy, 1978) controls rule selection by a set of metarules that help it deduce which task-oriented rules are likely to lead to a solution during the backward chaining search. Joshi (1978) approaches the problem of inference based on partial information by permitting only a small amount of data in the active portion of the data base at one time and then firing the collection of rules that best "accounts for" or matches this active data. Thus rules that only partially match the data (have some unmatched predicates in the antecedent) can be accessed. Representing rules in a semantic net

structure (Duda et al., 1978; Shapiro, 1977) has facilitated the use of subset and inclusion relations and provides a way to include the same relation in several different premises and conclusions.

Knowledge Acquisition

In the area of knowledge acquisition, considerable recent progress has been made. TEIRESIAS (Davis, 1976, 1977, 1978; Davis & Buchanan, 1977) facilitates the transfer of knowledge from a human expert to the knowledge base of a task-oriented system by applying general data structure schemata to construct new, specific inference rules. Waterman (1977a) has developed techniques by which man-machine interface programs can acquire the rules they need to perform personalized tasks for individual users, such as tutoring or creating reactive messages. A number of rule-based systems are able to infer new rules. Meta-DENDRAL contains the expertise of an organic chemist and is able to infer rules of fragmentation of molecules in a mass spectrometer (Buchanan & Fennell, 1977; Mitchell, 1977). Hayes-Roth and McDermott (1977) induce condition-action rules from examples by partial-matching the examples and generalizing their identified commonalities. Vere's approach is similar but separates rule antecedents into two parts, a static context and a modifiable premise, and generalizes each part differently when producing one general rule from many specific ones (Vere, 1977a, 1977b, 1978). Other approaches include applying generalization heuristics to

variable-valued logic decision rules (Larson & Michalski, 1977), generalizing examples of simple numerical patterns to produce rules relating the patterns to desired responses (Brown, 1975, 1977) and using feedback to evolve rules that enable a learning system to adapt best in an artificial environment (Holland & Reitman, 1978). Additional uses of partial matching techniques (Hayes-Roth, 1978; Hayes-Roth & McDermott, 1977) have been made in rule-based systems for inferring new concepts, pattern templates, syntactic categories, and problem-solving rules.

Knowledge Engineering

Knowledge engineering has been defined as "bringing the principles and tools of AI research to bear on difficult applications problems requiring experts' knowledge for their solution" (Feigenbaum, 1977). Programs constructed in this knowledge engineering framework embody domain-specific knowledge from an expert and apply it to produce useful inferences for the user. Much current work of this sort exploits a pattern-directed or rule-based organization.

A good example of an "expert" knowledge system is DENDRAL (Feigenbaum, Buchanan, & Lederberg, 1971), a performance program developed for the analysis of molecular structure. It makes use of mass spectrometry rules formulated by Meta-DENDRAL (Buchanan & Mitchell, 1978; Mitchell, 1977) and by experts. MOLGEN, a knowledge-based system for experiment planning in molecular genetics, uses data schemata to represent

procedural knowledge. Instances of schemata act as rules expressing transformational and problem solving knowledge (Martin et al., 1977). The MYCIN system (Shortliffe, 1976; Davis, Buchanan, & Shortliffe, 1977) contains rules for inferring the diagnosis of blood infections and recommending appropriate treatment. Expert knowledge is acquired via a related system called TEIRESIAS (Davis, 1977), which permits the user to interact with the knowledge system, adding new rules or modifying old ones. Other pattern-directed knowledge systems in medicine include IRIS, a medical consultation system that uses semantic nets and decision tables to make proper diagnoses (Trigoboff, 1976; Trigoboff & Kulikowski, 1977), and KAMM (Walser & McCormick, 1977), a program that accepts rules about management of chorioretinal diseases in ophthalmology and integrates them into the MEDICO system (Walser & McCormick, 1976), a knowledge-based consultant organized as a production system.

The use of multiple levels of abstraction in pattern interpretation has been the key to success in many recent projects, knowledge-based systems including the SU/X program, which takes continuous acoustic signal data and tries to identify their sources, and SU/P (also known as CRYSLIS), which infers 3-D models of protein molecules from an electron density map of crystallized protein (Nii & Feigenbaum, 1978). These programs use a global data base (blackboard) for integration of relevant knowledge and a multilevel representation of the partial solution hypotheses. Multiple

knowledge sources, organized as sets of rules, are used to generate inferences about both the solution and the overall strategy for applying the knowledge sources to various aspects of the problem. The form of multilevel knowledge representation in both programs was patterned after the Hearsay-II speech understanding system, which embodied a variety of independent, cooperating, pattern-directed knowledge sources (Hayes-Roth & Lesser, 1977; Hayes-Roth, Mostow, & Fox, 1977; Lesser & Erman, 1977). One particular knowledge source, a production system for syntax and semantics, was abandoned because of its inability to control the combinatorial explosion arising from partial matching of conditions by incomplete and errorful data (Hayes-Roth, 1978; Mostow & Hayes-Roth, 1978). Other recent work adopting similar multilevel architectures for pattern-directed inference includes a system that acquires knowledge about action-oriented competitive games (e.g., baseball) in the form of schemata that act as rules for interpreting and predicting future events (Soloway & Riseman, 1977), a system that constructs intentional interpretations of an observed sequence of human actions (Sridharan & Schmidt, 1978), and a system for playing Go that recognizes and updates pattern interrelations and uses this knowledge to drive high-level planning and problem-solving functions (Reitman & Wilcox, 1978).

Many other expert knowledge-based systems are now under development that use a rule-based formalism for knowledge representation. EL, an electrical network analysis program, has

been implemented in ARS, a problem-solving language employing rules as demons that monitor an associative data base (Sussman & Stallman, 1975; Sussman, 1977; Stallman & Sussman, 1976). Deductions are made by backward chaining, as in MYCIN, and the sequence of rules used in a deductive chain can be examined by the user. PONTIUS-0 (Goldstein & Grimson, 1977) is a production system for achieving skill in attitude instrument flying, where the goal is to maintain steady climbs, turns, descents, or level flying. This system uses "annotated" production rules, i.e., each rule is associated with a formal commentary describing caveats, rationales, plans, and control information. Other tasks performed by expert knowledge-based systems include automatic theory formation in mathematics (Lenat, 1976, 1977a), consultation about mineral exploration (Duda et al., 1978), business consultation (Mark, 1977), air traffic control (Wesson, 1977), information management in criminal court proceedings (Buchanan & Fennell, 1977), and modeling foreign policy decisionmaking (Sylvan, Thorson, & Thorson, 1977; Miller & Thorson, 1976).

Natural Language Understanding

Research in natural language understanding is now moving toward rule-based formalisms for representing knowledge about goals, motivation, and causality. A recent version of PAM (Schank & Wilensky, 1978), a system for story understanding, incorporates knowledge about the goals of the story characters as a production system that can build a structure in memory

describing the points (dramatic events) of the story in terms of a problem and its solution. The production rules are used primarily to fill the gaps in the meaning representation that the understander is trying to build. ELI, the English language interpreter used by the SAM and PAM story understanding systems, is organized as a production system that maps input text into a conceptual dependency structure. Riesbeck (1978) describes the system and discusses mechanisms used to reduce the problems caused by the use of large numbers of production rules. Schank (1977) uses condition-action rules to map sentences given to children into conceptual dependency representations of the meaning that the sentences have to the children. This is part of a system for modeling the linguistic development in a child from one to two years of age. A production system is also used to map sentences into a semantic network in a system designed to infer the meaning of phrases from the constituent words (McDonald & Hayes-Roth, 1978). Simmons (1978) uses rules to transform English discourse describing a story into semantic case predicates, then applies causal rules to these predicates to produce a net of causal relations among elements of the story, and finally is able to apply generative rules to such internal representations to produce natural language texts. Faught (1978) discusses a rule-based model of conversational action that describes and predicts speech acts in natural language dialogue. The model makes use of overlapping and parallel rules to predict and generate overlapping linguistic events and uses the emotional context of the dialogue to help resolve the ambiguous

interpretations that arise.

Cognitive Modeling

In the area of cognitive modeling based on pattern-directed inference, current work is heavily influenced by the earlier work of Newell (1972, 1973) with PSG. Young (1977) has developed production system models of children's behavior in seriation and subtraction tasks. The models use PSG-like production rules and attack the problem of representing a mixture of problem-solving strategies within a single integrated production system such that each strategy maintains its uniqueness as a separate collection of rules. Klahr and Wallace (1976) use PSG-like production systems to model cognitive development, in particular, tasks associated with Piaget's concrete operational stage. Models are produced that account for task behavior within a particular substage, and these are analyzed to determine how they could be modified to account for behavior at the next substage.

Another current approach to modeling human memory is embodied in the ACT system (Anderson, 1976; Anderson & Kline, 1977; Anderson, Kline, & Lewis, 1978). Memory is a propositional network of interconnected nodes, a small portion of which may be active at any one time. Operating on this memory is a set of production rules whose conditions test for the existence of certain features in the active portion of memory and whose actions can change the memory structure. Linear growth in computation time with the size of the data base or the number of

production rules is avoided by only considering data in the active part of memory when rules are applied and by associating with each node a list of rules that reference the node. When the node is activated the associated rules are selected for consideration. Thus the number of rules being tested against memory at any time depends on the number of currently active nodes rather than the size of the production system. Faught (1977) extended his use of production rules in a conversational action paradigm to include a model of affect (emotion) activation and response. The affect levels are used in the left-hand sides of production rules to provide a context for interpreting situations that the model is in (e.g., threatening situations) and in the right-hand sides of rules to effect responses to situations. Hayes-Roth (1978) evaluates human pattern processing and its relation to artificial knowledge systems in the light of recent psychological studies. Thorndyke (1978) describes a view of human text comprehension, memory, and recall that assumes information processing is guided by hierarchical patterns of abstract conceptual relations.

CONCLUSIONS

Pattern-directed inference systems have three fundamental characteristics. First, they are data- or event-driven, and thus respond directly to observed patterns in the environment. Second, there is modularity of control knowledge; control of the way the pattern components make contact with the data can be

described independently of the structure and function of the task knowledge. Finally, there is modularity of task knowledge. Both the knowledge that constitutes the data and knowledge that constitutes the heuristics for manipulating the data are organized into separate, although not necessarily independent, units (e.g., data elements, rules) that can be accessed, applied, or manipulated with some degree of autonomy. This simplifies the problem of isolating useful units of knowledge and facilitates the development of learning systems.

Two factors underlying the desirability of employing PDISs are environmental complexity and the diversity of the responses it necessitates. When intelligent behavior consists of numerous specialized responses to widely varying and largely unpredictable situations, the antecedent-consequent structure of PDMs isolates and represents the appropriate logic of such data-directed behavior in a natural way. In some cases, the complexity and scope of individual PDMs, the granularity of the PDIS, may be an option in the design and implementation of the system. It is important that the grain size match the problem domain characteristics. If the PDIS is too fine-grained for the particular task it may be laborious to program and difficult to understand. If the grain size is too coarse the PDIS may not perform as well as possible because of its nondiscriminating nature. Often the adoption of fine-grained, separable PDMs, so characteristic of production systems, leads to individual rules that can be induced from examples,

separately evaluated and verified, and even explained or understood at a local level. Of course, it is sometimes preferable to employ PDMs that are individually so complex that these putative benefits are not realizable. In exchange, the more coarsely segmented system is often easier to comprehend in toto since individual PDMs can be interpreted as more macroscopic behavior generators. Ultimately, the choice of grain-size should reflect an attempt to minimize the disparity between the task characteristics and the chosen problem representation.

Although pattern-directed inference systems have been in use and under development for a number of years, only recently has the range of task domains expanded to encompass many areas of artificial intelligence. The current interest in these systems and the success to date are good indications that this formalism provides a promising framework for the development of intelligent inference systems.

REFERENCES

- Anderson, J. R. Language, memory, and thought. Hillsdale, N.J.: Erlbaum, 1976.
- Anderson, J. R. and Kline, P. J. Design of a production system for cognitive modeling. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 60-65.
- Anderson, J. R., Kline, P. and Lewis, C. A production system model for language processing. In P. Carpenter and M. Just (Eds.), Cognitive processes in comprehension, Hillsdale, N.J.: Erlbaum, in press.
- Anderson, R. H. The use of production systems in RITA to construct personal computer "agents." Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 23-28.
- Anderson, R. H. and Gillogly, J. J. Rand intelligent terminal agent (RITA): Design philosophy. R-1809-ARPA, The Rand Corporation, Santa Monica, 1976. (a)
- Anderson, R. H. and Gillogly, J. J. The Rand intelligent terminal agent (RITA) as a network access aid. AFIPS Proceedings, 1976, 45, 501-509. (b)
- Anderson, R. H., et al. RITA reference manual. R-1808-ARPA, The Rand Corporation, Santa Monica, California, 1977.

- Barstow, D. A knowledge base organization for rules about programming. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 18-22. (a)
- Barstow, D. A knowledge-based system for automatic program construction. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 382-388. (b)
- Barstow, D. and Kant, E. Observations on the interaction between coding and efficiency knowledge in the PSI program synthesis system. Second International Conference on Software Engineering, San Francisco, 1976.
- Biermann, A. W. and Smith, D. R. A production rule mechanism for generating LISP code. Department of Computer Science, Duke University, 1977.
- Bobrow, D. G. and Winograd, T. An overview of KRL, a knowledge representation language. Cognitive Science, 1977, 1, 3-46.
- Bobrow, D. G., et al. Experience with KRL-0 one cycle of a knowledge representation language. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 213-222.
- Brooks, R. A model of human cognitive processes in writing code for computer programs. Ph.D. Dissertation, Department of Psychology, Carnegie-Mellon University, Pittsburgh, 1975.

- Brooks, R. Production systems as control structures for programming languages. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 33-37.
- Brown, D. J. H. Concept formation and exposition. Ph.D. Dissertation, University of the Witwatersrand, Johannesburg, South Africa, 1975.
- Brown, D. J. H. Concept learning by feature value interval abstraction. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 55-60.
- Buchanan, B. G. Scientific theory formation by computer. Proceedings of NATO Advanced Study Institute on Computer Oriented Learning Processes. Noordhoff, Leyden, 1976.
- Buchanan, B. G., Feigenbaum, E. A., and Sridharan, N. S. Heuristic theory formation: Data interpretation and rule formation. Machine Intelligence 7, Edinburgh University Press, 1972.
- Buchanan, B. G. and Mitchell, T. Model directed learning by production rules. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.
- Buchanan, B. G., Sutherland, G., and Feigenbaum, E. Heuristic DENDRAL: A program for generating explanatory

hypotheses in organic chemistry. In B. Meltzer and D. Michie (Eds.), Machine intelligence 4. New York: Elsevier, 1969, 209-254.

Buchanan, J. R. and Fennell, R. D. An intelligent information system for criminal case management in the federal courts. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 901-902.

Chomsky, A. N. Syntactic structures. Mouton, The Hague, 1957.

Davis, R. Application of meta level knowledge to the construction, maintenance and use of large knowledge bases. Memo AIM-283, Artificial Intelligence Laboratory, Stanford University, Stanford, 1976.

Davis R. Interactive transfer of expertise: Acquisition of new inference rules. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 321-328.

Davis, R. Knowledge acquisition in rule-based systems: Knowledge about representation as a basis for system construction and maintenance. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Davis R. and Buchanan B. G. Meta level knowledge: Overview and implications. Proceedings of the Fifth International Joint Conference on Artificial

Intelligence, Cambridge, 1977, 920-927.

Davis, R., Buchanan, B., and Shortliffe, E. Production rules as a representation for a knowledge-based consultation program.

Journal of Artificial Intelligence, 1977, 8, 15-45.

deKleer, J., et al. AMORD: Explicit control of reasoning.

Proceedings of the Symposium on Artificial Intelligence and Programming Languages, SIGPLAN Notices 12, SIGART Newsletter 64, 1977, 116-125.

Duda, R. O., et al. Semantic network representations in rule-based inference systems. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Farber, D. J., Griswold, R. E., and Polonsky, I. P. SNOBOL, a string manipulation language. Journal of the ACM, 1964, 11, 21-30.

Faught, W. S. Motivation and intensionality in a computer simulation model. AIM Memo 305, Stanford Artificial Intelligence Laboratory, Stanford, 1977.

Faught, W. S. Conversational action patterns in dialogs. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Feigenbaum, E. A. The art of artificial intelligence: I. Theme and case studies of knowledge engineering. Proceedings of the Fifth International Joint Conference on Artificial

Intelligence, Cambridge, 1977, 1014-1029.

Feigenbaum, E. A., Buchanan, B. G., and Lederberg, J. On generality and problem solving: A case study using the DENDRAL program. In B. Meltzer and D. Michie (Eds.), Machine intelligence 6. New York: Elsevier, 1971, 165-190.

Floyd, R. W. An algorithm for coding efficient arithmetic operations. Communications of the ACM, 1961, 4, 42-51.

Forgy, C. A production system monitor for parallel computers. Technical Report. Department of Computer Science, Carnegie-Mellon University, Pittsburgh, 1977.

Forgy, C. and McDermott, J. OPS, a domain-independent production system language. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 933-939.

Friend, K. An information processing approach to small group interaction in a coalition formation game. Psychology Department, Carnegie-Mellon University, Pittsburgh, 1973.

Galler, B. and Perlis, A. A view of programming languages. Addison-Wesley, 1970.

Goldstein, I. P. and Grimson, E. Annotated production systems: A model for skill acquisition. Proceedings of the Fifth International Joint Conference on Artificial

Intelligence, Cambridge, 1977, 311-317.

Green, C. The design of the PSI program synthesis system.
Second International Conference on Software
Engineering, San Francisco, 1976, 4-18.

Hayes-Roth, B. Implications of human pattern processing for the
design of artificial knowledge systems. In D. A. Waterman
and F. Hayes-Roth (Eds.), Pattern-directed inference
systems. New York: Academic Press, 1978.

Hayes-Roth, F. Patterns of induction and associated knowledge
acquisition algorithms. In C. H. Chen (Ed.), Pattern
recognition and artificial intelligence. New York: Academic
Press, 1976. (a)

Hayes-Roth, F. Uniform representations of structured patterns
and an algorithm for the induction of contingency-response
rules. Information and Control, 1976, 33, 87-116. (b)

Hayes-Roth, F. The role of partial and best matches in knowledge
systems. In D. A. Waterman and F. Hayes-Roth (Eds.),
Pattern-directed inference systems. New York: Academic
Press, 1978.

Hayes-Roth F. and Lesser, V. Focus of attention in the Hearsay
II speech understanding system. Proceedings of the Fifth
International Joint Conference on Artificial Intelligence,
Cambridge, 1977, 27-35.

Hayes-Roth, F. and McDermott, J. Knowledge acquisition from

structural descriptions. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977.

Hayes-Roth, F. and Mostow, D. J. An automatically compilable recognition network for structured patterns. Proceedings of the Fourth International Joint Conference on Artificial Intelligence, Tbilisi, Georgia, USSR, 1975, 246-251.

Hayes-Roth, F. and Mostow, D. J. Syntax and semantics in a distributed logic speech understanding system. Proceedings of the 1976 IEEE International Conference on Acoustics, Speech and Signal Processing, 421-424.

Hayes-Roth, F., Mostow, D. J., and Fox, M. S. Understanding speech in the Hearsay-II system. In L. Bolc (Ed), Natural language communication with computers. Berlin: Springer-Verlag, 1977.

Hayes-Roth, F., Waterman, D. A., and Lenat, D. Principles of pattern-directed inference systems. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Hedrick, C. L. A computer program to learn production systems using a semantic net. Ph.D. Dissertation, Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh, Pennsylvania, 1974.

Hedrick, C. L. Learning production systems from examples. Artificial Intelligence 7, 1976, 21-49.

- Hewitt, C. Procedural embedding of knowledge in PLANNER. Proceedings of the Second International Joint Conference on Artificial Intelligence, 1971, 167-184.
- Holland, J. H. and Reitman, J. S. Cognitive systems based on adaptive algorithms. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.
- Hunt, E. B. and Poltrock, S. E. The mechanics of thought. In B. H. Kantowitz (Ed.), Human information processing: Tutorials in performance and cognition. Hillsdale, N.J.: Lawrence Erlbaum Assoc., 1974, 277-350.
- Joshi, A. K. Some extensions of a system for inferencing on partial information. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.
- Kellogg, C., Klahr, P., and Travis, L. Deductive methods for large data bases. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 203-209.
- Kibler, D. F., Neighbors, J. M., and Standish, T. A. Program manipulation via an efficient production system. Proceedings of the Symposium on Artificial Intelligence and Programming Languages, SIGPLAN Notices 12, SIGART Newsletter 64, 1977, 163-173.
- Klahr, D. A production system for counting, subitizing and

adding. In W. G. Chase (Ed.), Visual information processing. New York: Academic Press, 1973.

Klahr, D. and Wallace, J. G. Cognitive development: An information processing view. Hillsdale, N.J.: Lawrence Erlbaum Associates, 1976.

Klahr, P. Planning techniques for rule selection in deductive question answering. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Larson, J. and Michalski, R. S. Inductive inference of VL decision rules. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 38-44.

Lenat, D. AM: An artificial intelligence approach to discovery in mathematics as heuristic search. SAIL AIM-286, Artificial Intelligence Laboratory, Stanford University, 1976. Jointly issued as Computer Science Dept. Report No. STAN-CS-76-570.

Lenat, D. Automated theory formation in mathematics. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 833-842. (a)

Lenat, D. The ubiquity of discovery: 1977 Computers and Thought Lecture. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge,

1977, 1093-1105. (b)

Lenat, D. and Harris, G. Designing a rule system that searches for scientific discoveries. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Lenat, D. and McDermott, J. Less than general production system architectures. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 928-932.

Lesser, V. R. and Erman, L. D. A retrospective view of the Hearsay-II architecture. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 790-800.

Manna, Z. and Waldinger, R. The automatic synthesis of systems of recursive programs. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 405-411.

Mark, W. S. The reformulation approach to building expert systems. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 329-335.

Markov, A. A. Theory of algorithms. National Academy of Sciences, USSR, 1954.

Martin, N., et al. Knowledge base management for experiment

planning in molecular genetics. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 882-887.

McCune, B. P. The PSI program model builder, synthesis of very high-level programs. Proceedings of the Symposium on Artificial Intelligence and Programming Languages, SIGPLAN Notices, 12, SIGART Newsletter 64, 1977, 130-139.

McDermott, D. A deductive model of control of a problem solver. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 2-7. (a)

McDermott, D. Vocabularies for problem solver state descriptions. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 229-234. (b)

McDermott, J. and Forgy, C. Production system conflict resolution strategies. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

McDermott, J., Newell, A., and Moore, J. The efficiency of certain production system implementations. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

McDonald, D. and Hayes-Roth, F. Inferential searches of knowledge networks as an approach to extensible language understanding systems. In D. A. Waterman and F. Hayes-Roth

(Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Miller, R. A. and Thorson, S. J. Control concepts in theories of governmental decision making: A structural approach. In J. V. Gillespie and D. J. Zinnes (Eds.), Mathematical systems in international relations research. New York: Praeger, 1976.

Minsky, M. Computation: Finite and infinite machines. Englewood Cliffs, N.J.: Prentice-Hall, 1967.

Mitchell, T. M. Version spaces: A candidate elimination approach to rule learning. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 305-310.

Moran, T. P. The symbolic imagery hypothesis: An empirical investigation via a production system simulation of human behavior in a visualization task. Ph.D. Dissertation, Department of Computer Science, Carnegie-Mellon University, 1974.

Mostow, D. J. and Hayes-Roth, F. A production system for speech understanding. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Newell, A. Studies in problem solving: Subject 3 on the cryptarithmic task: DONALD + GERALD = ROBERT. Carnegie-Mellon University, Pittsburgh, 1967.

- Newell, A. On the analysis of human problem solving protocols.
In J. C. Gardin and B. Jaulin (Eds.), Calcul et formalisation dans les sciences de l'homme. Paris: Centre National de la Recherche Scientifique, 1968, 146-185.
- Newell, A. A theoretical exploration of mechanisms for coding the stimulus. In A. W. Melton and E. Martin (Eds.), Coding processes in human memory. Washington, D.C.: Winston and Sons, 1972, 373-434.
- Newell, A. Production systems: Models of control structures. In W. G. Chase (Ed.), Visual information processing. New York: Academic Press, 1973, 463-526.
- Newell, A. and McDermott, J. PSG manual. Department of Computer Science, Carnegie-Mellon University, Pittsburgh, 1975.
- Newell, A. and Simon, H. A. An example of human chess play in the light of chess playing programs. In N. Wiener and J. P. Schade (Eds.), Progress in biocybernetics. Amsterdam: Elsevier Publishing Company, 1965.
- Newell, A. and Simon, H. A. Human problem solving. Englewood Cliffs, N.J.: Prentice-Hall, 1972.
- Nii, H. P. and Feigenbaum, E. A. Rule-based understanding of signals. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

- Post, E. L. Formal reductions of the general combinatorial decision problem. American Journal of Mathematics, 1943, 65, 197-268.
- Reitman, W. R. and Wilcox, B. Pattern recognition and pattern-directed inference in a program for playing Go. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.
- Rhyme, J. R. On finding conflict sets in production systems. Working Note, Department of Computer Science, University of Houston, 1977.
- Rieger, C. Spontaneous computation in cognitive models. Technical Report TR-459, Computer Science Department, University of Maryland, 1976.
- Rieger, C. and Grinberg, M. The declarative representation and procedural simulation of causality in physical mechanisms. TR-513, University of Maryland, 1977.
- Rieger, C. and London, P. Subgoal protection and unravelling during plan synthesis. TR-512, University of Maryland, 1977.
- Riesbeck, C. K. An expectation-driven production system for natural language. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.
- Rosenschein, S. J. The production system: Architecture and

abstraction. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Rulifson, J. F., Derksen, J. A., and Waldinger, R. J. QA4: A procedural calculus for intuitive reasoning. Stanford Research Institute, Menlo Park, California, 1972.

Rychener, M. D. Production systems as a programming language for artificial intelligence applications. Technical Report, Department of Computer Science, Carnegie-Mellon University, Pittsburgh, 1976.

Rychener, M. D. and Newell, A. An instructable production system: Basic design issues. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Schank, R. C. and Selfridge, M. How to learn/What to learn. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 8-14.

Schank, R. C. and Wilensky, R. A goal-directed production system for story understanding. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Shapiro, S. C. Representing and locating deduction rules in a semantic network. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63,

1977, 14-18.

Shortliffe, E. H. Computer-based medical consultations: MYCIN.
New York: American Elsevier, 1976.

Shortliffe, E. H., et al. An artificial intelligence program to
advise physicians regarding antimicrobial therapy.
Computers and Biomedical Research, 1973, 6, 544-560.

Shortliffe, E. H., et al. Computer-based consultations in
clinical therapeutics: Explanation and rule
acquisition capabilities of the MYCIN system. Computers
and Biomedical Research, 1975, 8, 303-320.

Simmons, R. F. Rule-based computations on English. In D. A.
Waterman and F. Hayes-Roth (Eds.), Pattern-directed
inference systems. New York: Academic Press, 1978.

Smith, D. H., Masinter, L. M., and Sridharan, N. S. Heuristic
DENDRAL: Analysis of molecular structure. Proceedings of
the NATO/CNNA Advanced Study Institute on Computer
Representation and Manipulation of Chemical Information,
1974.

Smith, M. J. and Sleeman, D. H. APRIL: A flexible production
rule interpreter. Proceedings of the Workshop on
Pattern-Directed Inference Systems, SIGART Newsletter 63,
1977, 28-33.

Soloway, E. M. and Riseman, E. M. Levels of pattern description
in learning. Proceedings of the Fifth International

Joint Conference on Artificial Intelligence, Cambridge, 1977, 801-811.

Sridharan, N. S. and Schmidt, C. F. Knowledge-directed inference in BELIEVER. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Stallman, R. and Sussman, G. J., Forward reasoning and dependency-directed backtracking in a system for computer-aided circuit analysis. MIT Artificial Intelligence Laboratory Memo 380, Cambridge, 1976.

Sussman, G. J. Electrical design: A problem for artificial intelligence research. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 894-900.

Sussman, G. and Stallman, R. Heuristic techniques in computer aided circuit analysis. Memo 328, MIT AI Lab., Cambridge, 1975.

Sylvan, D. A., Thorson, E., and Thorson, S. J. Processing styles and theories of foreign policy decision making. Department of Political Science, Ohio State University, 1977.

Szolovits, P., Hawkinson, L. B., Martin, W. A. An overview of OWL, a language for knowledge representation. MIT/LCS/TM-86, Laboratory for Computer Science, MIT, 1977.

Thorndyke, P. W. Pattern-directed processing of knowledge from

texts. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Travis, L., et al. Design rationale for TELOS, a PASCAL-based AI language. Proceedings of the Symposium on Artificial Intelligence and Programming Languages, SIGPLAN Notices 12, SIGART Newsletter 64, 1977, 67-76.

Trigoboff, M. Propagation of information in a semantic net. Proceedings AISB Summer Conference, Edinburgh, Scotland, 1976, 334-343.

Trigoboff, M. and Kulikowski, C. A. IRIS: A system for the propagation of inferences in a semantic net. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 274-280.

Vere, S. A. Relational production systems. Artificial Intelligence, 1977, 8, 47-68. (a)

Vere, S. A. Induction of relational productions in the presence of background information. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, 1977, 349-355. (b)

Vere, S. A. Inductive learning of relational productions. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Walser, R. L. and McCormick, B. H. Organization of clinical

knowledge in MEDICO. Proceedings of the Third Illinois Conference on Medical Information Systems, 1976.

Walser, R. L. and McCormick, B. H. A system for priming a clinical knowledge base. Proceedings of the National Computer Conference, 1977, 301-307.

Waterman, D. A. Machine learning of heuristics. Ph.D. Dissertation, Computer Science Department, Stanford University, 1968.

Waterman, D. A. Generalization learning techniques for automating the learning of heuristics. Artificial Intelligence, 1970, 1, 121-170.

Waterman, D. A. Adaptive production systems. Proceedings of the Fourth International Joint Conference on Artificial Intelligence, Tbilisi, Georgia, USSR, 1975, 1, 296-303.

Waterman, D. A. Serial pattern acquisition: A production system approach. In C. H. Chen (Ed.), Pattern recognition and artificial intelligence. New York: Academic Press, 1976.
(a)

Waterman, D. A. An introduction to production systems. Rand Paper P-5751, Rand Corporation, Santa Monica, 1976. (b)

Waterman, D. A. A rule-based approach to knowledge acquisition for man-machine interface programs. Rand Paper P-5895, Rand Corporation, Santa Monica, 1977. (a)

Waterman, D. A. Rule-directed interactive transaction agents:

An approach to knowledge acquisition. Rand Report R-2171-ARPA, Rand Corporation, Santa Monica, 1977. (b)

Waterman, D. A. Exemplary programming in RITA. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Waterman, D. A. and Newell, A. Protocol analysis as a task for artificial intelligence. Artificial Intelligence, 1971, 2, 285-318.

Wesson, R. B. Planning in the world of the air traffic controller. Proceedings of the Fifth International Joint Conference on Artificial Intelligence, Cambridge, 1977, 473-479.

Yngve, V. H. A programming language for mechanical translation. Mechanical Translation, 1958, 5, 25-41.

Young, R. Children's seriation behavior: A production system analysis. Ph.D. Dissertation, Psychology Department, Carnegie-Mellon University, Pittsburgh, 1973.

Young, R. M. Mixtures of strategies in structurally adaptive production systems: Examples from seriation and subtraction. Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63, 1977, 65-71.

Zisman, M. D. A representation for office processes. Working Paper 76-10-03, Department of Decision Sciences, The Wharton

School, University of Pennsylvania, 1976.

Zisman, M. D. Use of production systems for modeling asynchronous, concurrent processes. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.

Zucker, S. W. Production systems with feedback. In D. A. Waterman and F. Hayes-Roth (Eds.), Pattern-directed inference systems. New York: Academic Press, 1978.